C++ Programming Examples: A Comprehensive Guide Based on Balagurusamy

Introduction

This book provides a detailed explanation of C++ programming concepts through practical examples drawn from Balagurusamy's classic textbook. Each program is analyzed step-by-step, with explanations of the underlying concepts, code structure, and execution flow.

The examples cover fundamental C++ topics including:

- Basic programming constructs
- Object-oriented programming principles
- Memory management
- String manipulation
- Data structures

Table of Contents

- Class Fundamentals
- Member Functions
- Arrays of Objects
- Static Members

6. Constructors and Destructors

- Default Constructors
- Parameterized Constructors
- Copy Constructors
- Destructor Concepts

7. Inheritance

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Virtual Base Classes

8. Operator Overloading

- Unary Operators
- Binary Operators
- Friend Functions

9. Dynamic Memory Management

- malloc
- calloc

Chapter 1: Basic C++ Concepts

1.1 Inline Functions

Inline functions are a feature in C++ that allows the compiler to expand the function call inline at the point where it is called, rather than performing a function call. This can improve performance for small functions by eliminating the overhead of function calls.

Program: Inline function.cpp

```
#include <iostream>

using namespace std;

inline int max(int a, int b)
{
    return (a > b) ? a : b;
}

int main()
{
    int x = 10, y = 20;
    cout << "Maximum value: " << max(x, y);
    return 0;
}
```

Explanation:

- The `inline` keyword suggests to the compiler to replace the function call with the actual code.
- For small functions like `max`, this can be more efficient.
- The function compares two integers and returns the larger one.
- In `main`, we call `max` with arguments 10 and 20, which returns 20.

Output: Maximum value: 20

1.2 Reference Variables

Reference variables provide an alias for an existing variable. They must be initialized when declared and cannot be changed to refer to another variable.

Program: Reference variables.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 10;
    int &b = a;  // b is a reference to a
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    b = 20;  // Changing b changes a
    cout << "After changing b:" << endl;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    return 0;
}
```

Explanation:

- `int &b = a;` declares b as a reference to a.
- Any changes to b affect a and vice versa.
- References are useful for function parameters to avoid copying large objects.

Output: a = 10 b = 10 After changing b: a = 20 b = 20

1.3 Static Variables

Static variables retain their value between function calls and are initialized only once.

Program: Static variable.cpp

```
#include <iostream>

using namespace std;

void counter()
{
    static int count = 0;
    count++;
    cout << "Count: " << count << endl;
}
```

```
int main()
{
    counter();
    counter();
    counter();
    return 0;
}
```

Explanation:

- `static int count = 0;` initializes count to 0 only once.
- Each call to `counter()` increments the same variable.
- Without `static`, count would be reinitialized to 0 each time.

Output: Count: 1 Count: 2 Count: 3

1.4 External Variables

External variables (global variables) can be accessed from any function in the program.

Program: Extern variable.cpp

```
#include <iostream>

using namespace std;

int globalVar = 100;  // Global variable

void display()
{
    cout << "Global variable: " << globalVar << endl;
}

int main()
{
    cout << "In main: " << globalVar << endl;
    display();
    globalVar = 200;
    display();
    return 0;
}
```

Explanation:

- `globalVar` is declared outside any function, making it global.
- It can be accessed and modified by any function.
- Global variables should be used sparingly as they can make code harder to maintain.

Output: In main: 100 Global variable: 100 Global variable: 200

1.5 Scope Resolution Operator

The scope resolution operator (::) is used to access global variables when a local variable with the same name exists.

Program: Scope resolution operator.cpp

```
#include <iostream>

using namespace std;

int x = 10;  // Global x

int main()
{
    int x = 20;  // Local x
    cout << "Local x: " << x << endl;
    cout << "Global x: " << ::x << endl;
    return 0;
}
```

Explanation:

- `::x` accesses the global variable x.
- Without `::`, the local x would be used.
- This operator is also used to define member functions outside the class.

Output: Local x: 20 Global x: 10

1.6 Preprocessors

Preprocessors are directives that are processed before compilation.

Program: The preprocessors.cpp

```
#include <iostream>
#define PI 3.14159
#define SQUARE(x) (x * x)

using namespace std;

int main()
{
    cout << "PI: " << PI << endl;
    cout << "Square of 5: " << SQUARE(5) << endl;
    return 0;
}
```

Explanation:

- `#define PI 3.14159` defines a constant.
- `#define SQUARE(x) (x * x)` defines a macro function.
- Preprocessor directives start with # and are processed before compilation.

Output: PI: 3.14159 Square of 5: 25

Chapter 2: Arrays and Pointers

2.1 Pointer Basics

Pointers store memory addresses of variables.

Program: POINTER - Display data of other variable using pointer.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int a = 1;
    int *p = &a;
    cout << "Data of (a): " << *p << endl;
    cout << "Address of (a): " << p;
}
```

Explanation:

- `int *p = &a;` declares p as a pointer to int and assigns address of a.
- `*p` dereferences the pointer to get the value.
- `p` gives the address.

Output: Data of (a): 1 Address of (a): [some address]

2.2 Pointer Arithmetic

Pointers can be incremented/decremented to navigate arrays.

Program: POINTER - Sum of all array elements.cpp

```
#include <iostream>

using namespace std;

int main()
{
    int arr[] = {1, 2, 3, 4, 5};
    int *p = arr;
    int sum = 0;
    for(int i = 0; i < 5; i++)
    {
        sum += *p;
        p++;
    }
    cout << "Sum: " << sum;
    return 0;
}
```

Explanation:

- `int *p = arr;` points to the first element.
- `p++` moves to the next element.
- `*p` accesses the current element.

Output: Sum: 15

2.3 Pointers to Functions

Functions can have pointers as parameters or return pointers.

Program: POINTER - Pointer to a function.cpp

```
#include <iostream>

using namespace std;

void display(int *p)
{
    cout << "Value: " << *p << endl;
}

int main()
{
    int a = 10;
    display(&a);
    return 0;
}
```

Explanation:

- `void display(int *p)` takes a pointer to int.
- `display(&a)` passes the address of a.
- Inside display, `*p` accesses a's value.

Output: Value: 10

2.4 Returning Pointers

Functions can return pointers to memory.

Program: POINTER - Functions returning pointers.cpp

```
#include <iostream>

using namespace std;

int* getPointer()
{
    static int x = 100;
    return &x;
}

int main()
{
    int *p = getPointer();
    cout << "Value: " << *p << endl;
    return 0;
}
```

Explanation:

- `int* getPointer()` returns a pointer to int.
- Returns address of static variable x.
- Caller can access the value via the returned pointer.

Output: Value: 100

Chapter 3: Strings and Character Handling

3.1 String Length

`strlen` counts characters in a string.

Program: STRLEN to count string data.cpp

```
#include <iostream>
#include <cstring>

using namespace std;

int main()
{
    char a[] = "Hello";
    char b[] = " World";
    cout << strlen(a) + strlen(b);
    return 0;
}
```

Explanation:

- `strlen(a)` returns 5 (length of "Hello").
- Note: Missing #include in original, but required for strlen.

Output: 11

3.2 String Copy

`strcpy` copies one string to another.

Program: STRCPY - Copy data of one string into other.cpp

```
#include <iostream>
#include <cstring>

using namespace std;

int main()
{
    char a[20] = "Hello";
    char b[] = " World";
    strcpy(a, b);
    cout << a;
    return 0;
}
```

Explanation:

- `strcpy(a, b)` copies " World" into a, overwriting "Hello".
- a must be large enough to hold the copied string.

Output: World

3.3 String Concatenation

`strcat` appends one string to another.

Program: STRCAT to concatenate two strings.cpp

```
#include <iostream>
#include <cstring>
```

```
using namespace std;

int main()
{
    char a[20] = "Hello";
    char b[] = " World";
    strcat(a, b);
    cout << a;
    return 0;
}
```

Explanation:

- `strcat(a, b)` appends b to a.
- a must have enough space for both strings.

Output: Hello World

3.4 String Comparison

`strcmp` compares two strings lexicographically.

Program: STRCMP - Comparing two strings.cpp

```
#include <iostream>
#include <cstring>

using namespace std;

int main()
{
    char a[] = "Apple";
    char b[] = "Banana";
    int result = strcmp(a, b);
    if(result < 0)
        cout << "a comes before b";
    else if(result > 0)
        cout << "b comes before a";
    else
        cout << "Strings are equal";
    return 0;
}
```

Explanation:

- `strcmp` returns negative if a < b, positive if a > b, 0 if equal.
- Comparison is case-sensitive.

Output: a comes before b

3.5 Character Input/Output

`getchar` and `putchar` handle single characters.

Program: GETCHAR & PUTCHAR - Getting and displaying character.cpp

```
#include <iostream>
```

```
using namespace std;

int main()
{
    char ch;
    cout << "Enter a character: ";
    ch = getchar();
    cout << "You entered: ";
    putchar(ch);
    return 0;
}
```

Explanation:

- `getchar()` reads a single character from input.
- `putchar(ch)` displays the character.
- Note: `getchar` is from , but works with cin/cout.

Output: Depends on user input.

Chapter 4: Structures and Unions

4.1 Structure Basics

Structures group related data.

Program: Structure basics.cpp

```
#include <iostream>

using namespace std;

struct Student
{
    int roll;
    char name[20];
    float marks;
};

int main()
{
    struct Student s = {1, "John", 85.5};
    cout << "Roll: " << s.roll << endl;
    cout << "Name: " << s.name << endl;
    cout << "Marks: " << s.marks << endl;
    return 0;
}
```

Explanation:

- `struct Student` defines a structure with roll, name, marks.
- `s` is a variable of type Student.
- Members accessed with dot operator.

Output: Roll: 1 Name: John Marks: 85.5

4.2 Arrays of Structures

Arrays can hold multiple structure instances.

Program: Structure - Array of structures.cpp

```
#include <iostream>

using namespace std;

struct Student
{
    int roll;
    char name[20];
    float marks;
};

int main()
{
    struct Student s[3] = {
        {1, "John", 85.5},
        {2, "Jane", 92.0},
        {3, "Bob", 78.3}
    };
    for(int i = 0; i < 3; i++)
    {
        cout << "Student " << i+1 << ":" << endl;
        cout << "Roll: " << s[i].roll << endl;
        cout << "Name: " << s[i].name << endl;
        cout << "Marks: " << s[i].marks << endl << endl;
    }
    return 0;
}
```

Explanation:

- `s[3]` declares array of 3 Student structures.
- Initialized with values for each student.
- Loop accesses each element.

Output: Details of 3 students.

4.3 Unions

Unions share memory among members.

Program: UNION.cpp

```
#include <iostream>

using namespace std;

union Data
{
    int i;
    float f;
    char str[20];
};

int main()
{
    union Data data;
    data.i = 10;
```

```
    cout << "Integer: " << data.i << endl;
    data.f = 3.14;
    cout << "Float: " << data.f << endl;
    strcpy(data.str, "Hello");
    cout << "String: " << data.str << endl;
    return 0;
}
```

Explanation:

- All members share the same memory location.
- Only one member can hold a value at a time.
- Size of union is size of largest member.

Output: Integer: 10 Float: 3.14 String: Hello

Chapter 5: Classes and Objects

5.1 Class Fundamentals

Classes encapsulate data and functions.

Program: Classes and Objects - Basics.cpp

```
#include <iostream>

using namespace std;

class Item
{
    private:
        int number;
        float cost;
    public:
        void getData(int, float);
        void putData(void);
};

void Item::getData(int a, float b)
{
    number = a;
    cost = b;
}

void Item::putData(void)
{
    cout<<"Number: "<<number<<endl;
    cout<<"Cost: "<<cost;
}

int main()
{
    Item item;
    item.getData(34, 78);
    item.putData();
    return 0;
}
```

Explanation:

- `class Item` defines a class with private data and public methods.
- `item` is an object of the class.
- Methods access and display the data.

Output: Number: 34 Cost: 78

5.2 Arrays of Objects

Objects can be stored in arrays.

Program: Classes and Objects - Array of objects.cpp

```
#include <iostream>

using namespace std;

class Item
{
    private:
        int number;
        float cost;
    public:
        void getData(int, float);
        void putData(void);
};

void Item::getData(int a, float b)
{
    number = a;
    cost = b;
}

void Item::putData(void)
{
    cout<<"Number: "<<number<<endl;
    cout<<"Cost: "<<cost<<endl;
}

int main()
{
    Item items[3];
    for(int i = 0; i < 3; i++)
    {
        items[i].getData(i+1, (i+1)*10.0);
        items[i].putData();
    }
    return 0;
}
```

Explanation:

- `items[3]` creates array of 3 Item objects.
- Each object is initialized and displayed separately.

Output: Details of 3 items.

5.3 Static Members

Static members belong to the class, not individual objects.

Program: Classes and Objects - Static data members.cpp

```
#include <iostream>

using namespace std;

class Item
{
    private:
        static int count;
        int number;
    public:
        void getData(int a)
        {
            number = a;
            count++;
        }
        void putData(void)
        {
            cout<<"Number: "<<number<<endl;
            cout<<"Count: "<<count<<endl;
        }
};

int Item::count = 0;

int main()
{
    Item i1, i2;
    i1.getData(10);
    i1.putData();
    i2.getData(20);
    i2.putData();
    return 0;
}
```

Explanation:

- `static int count;` shared among all objects.
- Incremented each time getData is called.
- Definition `int Item::count = 0;` outside class.

Output: Number: 10 Count: 1 Number: 20 Count: 2

Chapter 6: Constructors and Destructors

6.1 Default Constructor

Called automatically when object is created.

Program: Constructor and Destructor - Default constructor.cpp

```
#include <iostream>

using namespace std;

class Item
{
    private:
        int number;
```

```
    public:
        Item()  // Default constructor
        {
            number = 0;
            cout << "Default constructor called" << endl;
        }
        void display()
        {
            cout << "Number: " << number << endl;
        }
};

int main()
{
    Item item;
    item.display();
    return 0;
}
```

Explanation:

- `Item()` is the default constructor.
- Called when `Item item;` is executed.
- Initializes number to 0.

Output: Default constructor called Number: 0

6.2 Parameterized Constructor

Takes parameters to initialize objects.

Program: Constructor and Destructor - Parameterized constructor.cpp

```
#include <iostream>

using namespace std;

class Item
{
    private:
        int number;
    public:
        Item(int n)  // Parameterized constructor
        {
            number = n;
            cout << "Parameterized constructor called with " << n << endl;
        }
        void display()
        {
            cout << "Number: " << number << endl;
        }
};

int main()
{
    Item item(5);
    item.display();
    return 0;
}
```

Explanation:

- `Item(int n)` takes an integer parameter.
- Initializes number with the passed value.

Output: Parameterized constructor called with 5 Number: 5

6.3 Copy Constructor

Creates a copy of an existing object.

Program: Constructor and Destructor - Copy constructor.cpp

```
#include <iostream>

using namespace std;

class Item
{
    private:
        int number;
    public:
        Item(int n)
        {
            number = n;
        }
        Item(const Item &obj)  // Copy constructor
        {
            number = obj.number;
            cout << "Copy constructor called" << endl;
        }
        void display()
        {
            cout << "Number: " << number << endl;
        }
};

int main()
{
    Item item1(10);
    Item item2 = item1;  // Calls copy constructor
    item2.display();
    return 0;
}
```

Explanation:

- `Item(const Item &obj)` is the copy constructor.
- Called when `item2 = item1;` is executed.
- Copies the data from obj to the new object.

Output: Copy constructor called Number: 10

6.4 Destructor

Called when object goes out of scope.

Program: Constructor and Destructor - Destructor.cpp

```
#include <iostream>

using namespace std;
```

```
class Item
{
    private:
        int number;
    public:
        Item(int n)
        {
            number = n;
            cout << "Constructor called for " << number << endl;
        }
        ~Item()  // Destructor
        {
            cout << "Destructor called for " << number << endl;
        }
};

int main()
{
    Item item(5);
    cout << "End of main" << endl;
    return 0;
}
```

Explanation:

- `~Item()` is the destructor.
- Called automatically when item goes out of scope.
- Used for cleanup operations.

Output: Constructor called for 5 End of main Destructor called for 5

Chapter 7: Inheritance

7.1 Single Inheritance

A class inherits from one base class.

Program: Inheritance - Single (public).cpp

```
#include <iostream>

using namespace std;

class A
{
    private:
        int a = 5;
    public:
        int b;
        void putData(int x)
        {
            b = a * x;
        }
};

class B:public A
{
    private:
        int c = 10;
```

```
    public:
        int getData(void)
        {
            return (c * b);
        }
};

int main()
{
    B b;
    b.putData(30);
    cout << b.getData();
    return 0;
}
```

Explanation:

- `class B:public A` makes B inherit from A publicly.
- B can access public members of A.
- Private members of A are not accessible in B.

Output: 1500 (10 * (5 * 30))

7.2 Multiple Inheritance

A class inherits from multiple base classes.

Program: Inheritance - Multiple.cpp

```
#include <iostream>

using namespace std;

class A
{
    public:
        int a = 10;
};

class B
{
    public:
        int b = 20;
};

class C:public A, public B
{
    public:
        int getData()
        {
            return a + b;
        }
};

int main()
{
    C c;
    cout << c.getData();
    return 0;
}
```

Explanation:

- `class C:public A, public B` inherits from both A and B.
- C has access to members of both base classes.

Output: 30

7.3 Multilevel Inheritance

A class inherits from another derived class.

Program: Inheritance - Multilevel.cpp

```
#include <iostream>

using namespace std;

class A
{
    public:
        int a = 10;
};

class B:public A
{
    public:
        int b = 20;
};

class C:public B
{
    public:
        int getData()
        {
            return a + b;
        }
};

int main()
{
    C c;
    cout << c.getData();
    return 0;
}
```

Explanation:

- A -> B -> C forms a chain of inheritance.
- C inherits from B, which inherits from A.
- C has access to members of both A and B.

Output: 30

Chapter 8: Operator Overloading

8.1 Unary Operator Overloading

Overloading operators like ++, --.

Program: Operator overloading - Unary minus.cpp

```
#include <iostream>

using namespace std;

class Complex
{
    private:
        int real, imag;
    public:
        Complex(int r = 0, int i = 0)
        {
            real = r;
            imag = i;
        }
        Complex operator-()
        {
            return Complex(-real, -imag);
        }
        void display()
        {
            cout << real << " + " << imag << "i" << endl;
        }
};

int main()
{
    Complex c1(3, 4);
    Complex c2 = -c1;
    cout << "Original: ";
    c1.display();
    cout << "Negated: ";
    c2.display();
    return 0;
}
```

Explanation:

- `Complex operator-()` overloads the unary minus operator.
- Returns a new Complex with negated values.

Output: Original: 3 + 4i Negated: -3 + -4i

8.2 Binary Operator Overloading

Overloading operators like +, -, *.

Program: Operator overloading - Binary.cpp

```
#include <iostream>

using namespace std;

class Complex
{
    private:
        int real, imag;
    public:
        Complex(int r = 0, int i = 0)
        {
            real = r;
```

```
            imag = i;
        }
        Complex operator+(Complex const &obj)
        {
            Complex res;
            res.real = real + obj.real;
            res.imag = imag + obj.imag;
            return res;
        }
        void display()
        {
            cout << real << " + " << imag << "i" << endl;
        }
};

int main()
{
    Complex c1(3, 4), c2(1, 2);
    Complex c3 = c1 + c2;
    cout << "Sum: ";
    c3.display();
    return 0;
}
```

Explanation:

- `Complex operator+(Complex const &obj)` overloads +.
- Adds corresponding real and imaginary parts.

Output: Sum: 4 + 6i

8.3 Friend Functions

Friend functions can access private members.

Program: Operator overloading - Binary operator with friend function.cpp

```
#include <iostream>

using namespace std;

class Complex
{
    private:
        int real, imag;
    public:
        Complex(int r = 0, int i = 0)
        {
            real = r;
            imag = i;
        }
        friend Complex operator+(Complex const &, Complex const &);
        void display()
        {
            cout << real << " + " << imag << "i" << endl;
        }
};

Complex operator+(Complex const &c1, Complex const &c2)
{
    return Complex(c1.real + c2.real, c1.imag + c2.imag);
}
```

```
int main()
{
    Complex c1(3, 4), c2(1, 2);
    Complex c3 = c1 + c2;
    cout << "Sum: ";
    c3.display();
    return 0;
}
```

Explanation:

- `friend Complex operator+` declares + as a friend function.
- Friend function can access private members.
- Defined outside the class.

Output: Sum: 4 + 6i

Chapter 9: Dynamic Memory Management

9.1 malloc

Allocates memory dynamically.

Program: MALLOC - Dynamic memory allocation.cpp

```
#include <iostream>
#include <cstdlib>

using namespace std;

int main()
{
    int *p = (int*)malloc(sizeof(int));
    if(p == NULL)
    {
        cout << "Memory allocation failed" << endl;
        return 1;
    }
    *p = 10;
    cout << "Value: " << *p << endl;
    free(p);
    return 0;
}
```

Explanation:

- `malloc(sizeof(int))` allocates memory for one int.
- Returns void*, cast to int*.
- `free(p)` deallocates the memory.

Output: Value: 10

9.2 calloc

Allocates and initializes memory to zero.

Program: CALLOC.cpp

```
#include <iostream>
#include <cstdlib>

using namespace std;

int main()
{
    int *p = (int*)calloc(5, sizeof(int));
    if(p == NULL)
    {
        cout << "Memory allocation failed" << endl;
        return 1;
    }
    cout << "Values: ";
    for(int i = 0; i < 5; i++)
    {
        cout << p[i] << " ";
    }
    free(p);
    return 0;
}
```

Explanation:

- `calloc(5, sizeof(int))` allocates memory for 5 ints, initialized to 0.
- All elements are 0 initially.

Output: Values: 0 0 0 0 0

Conclusion

This book has covered the fundamental concepts of C++ programming through practical examples. Each program demonstrates a specific concept with detailed explanations. To master C++, practice implementing these concepts and experiment with variations.

For further learning, consider:

- Reading Balagurusamy's "Object Oriented Programming with C++"
- Practicing on online coding platforms
- Building small projects to apply these concepts

Remember that programming is best learned through hands-on practice. Happy coding!